

Informe — Evaluación Parcial N°3

CI/CD con Autoscaling y Load Balancer en AWS EKS

Aplicación «Tienda Perritos»¹

Integrantes (dupla):	Francisco Gómez Ramos · Benjamin Aravena Rosales
Fecha:	28 / 06 / 2026
Repositorio GitHub:	github.com/FranciscoGomezRa/tienda-perritos-eks
Región AWS / Clúster:	us-east-1 / tienda-eks
Namespace:	tienda

¹ Documento elaborado con apoyo de una herramienta de inteligencia artificial (Claude, de Anthropic) para la redacción y estructuración del texto. El contenido técnico, las decisiones de arquitectura y la evidencia fueron ejecutados y verificados por los integrantes.

1. Introducción

El presente informe documenta el desarrollo de la Evaluación Parcial N°3 de la asignatura **Introducción a Herramientas DevOps (ISY1101)**, cuyo propósito es llevar la aplicación web «**Tienda Perritos**» —una tienda en línea de productos para mascotas compuesta por un *frontend* (Nginx + JavaScript), un *backend* (Node.js / Express) y una base de datos **MySQL**— hacia un entorno de orquestación de contenedores productivo en la nube.

Para lograrlo se implementó un clúster de Kubernetes gestionado en **AWS EKS**, un pipeline de integración y despliegue continuo (**CI/CD**) con **GitHub Actions** que automatiza el ciclo *build* → *push* → *deploy*, autoescalado horizontal de réplicas (**HPA**) y balanceo de carga mediante un **Elastic Load Balancer**. El objetivo es demostrar despliegue automatizado, escalabilidad, alta disponibilidad, gestión segura de credenciales y observabilidad.

El documento se organiza siguiendo los siete indicadores de evaluación del encargo (IE1 a IE7). Cada indicador se acompaña de su justificación técnica, la evidencia gráfica real capturada durante la ejecución del proyecto y los comandos de *CloudShell* asociados, de modo que cada criterio de la rúbrica quede respaldado por una prueba verificable.

2. Contexto del Caso

Tienda Perritos es un comercio electrónico dedicado a la venta de productos para mascotas. Tras haber contenedorizado su aplicación en etapas previas (Docker) y montar una infraestructura base en AWS, el negocio enfrenta un nuevo desafío: su catálogo y su tráfico crecen, las campañas y los picos de demanda son cada vez más frecuentes, y una caída del sitio o un despliegue manual fallido se traducen directamente en ventas perdidas.

Frente a este escenario, el equipo técnico debe evolucionar desde «contenedores que corren en una máquina» hacia una **plataforma de orquestación productiva** capaz de:

- ejecutar la aplicación de forma **escalable**, agregando o quitando réplicas según la carga real;
- mantenerse **altamente disponible** y **tolerante a fallos**, repartiéndose en varias zonas de disponibilidad y autorrecuperándose ante la caída de un contenedor;
- **automatizar por completo los despliegues** desde GitHub, eliminando los pasos manuales propensos a error;
- **registrar logs y métricas** para diagnosticar problemas y medir el desempeño;

- y **proteger las credenciales** sensibles (acceso a AWS y a la base de datos) fuera del código fuente.

Este informe responde a ese contexto: presenta la arquitectura diseñada para Tienda Perritos sobre AWS EKS y demuestra, con evidencia, que la solución cumple cada uno de esos objetivos de negocio traducidos en los indicadores técnicos IE1 a IE7.

3. Propuesta de Solución

La solución propuesta orquesta los tres servicios de Tienda Perritos en un clúster **Amazon EKS** (tienda-eks, región us-east-1, namespace tienda), desplegando las imágenes desde un registro privado **Amazon ECR** y automatizando todo el ciclo de vida con **GitHub Actions**. Sus componentes principales son:

Infraestructura de red (VPC)

- VPC **tienda-vpc** (10.0.0.0/16) distribuida en **2 zonas de disponibilidad** (us-east-1a y us-east-1b) para alta disponibilidad.
- **6 subredes**: 2 públicas (alojan el Load Balancer y el NAT Gateway) y 4 privadas (alojan los nodos worker, sin IP pública).
- **NAT Gateway** para que los nodos privados tengan salida a Internet (por ejemplo, para hacer *docker pull* desde ECR) sin quedar expuestos.

Cómputo y orquestación

- Clúster **EKS** con plano de control gestionado por AWS y un **node group** de instancias t3.medium (2/2/4) en las subredes privadas.
- Tres *Deployments*: frontend, backend y base de datos, cada uno con sus *requests/limits* de CPU y sondas de salud (*readiness/liveness*).
- **HPA** (Horizontal Pod Autoscaler) sobre frontend y backend, alimentado por el complemento *Metrics Server* del clúster.

Conectividad y balanceo

- Service **tienda-frontend** de tipo LoadBalancer → expone la app por un **Classic ELB** público (URL accesible desde el navegador).
- Services internos tienda-backend y tienda-db resueltos por **DNS interno** de Kubernetes.

Flujo de la petición del usuario

Usuario → **ELB** (subred pública) → Service/Pods **frontend** (Nginx) → Service **backend** (DNS interno tienda-backend) → Pods **backend** (Node/Express) → Service **db** (tienda-db) → **MySQL**.

Flujo CI/CD

commit a la rama **main** → **GitHub Actions** (Environment *production*) → *build* de las imágenes → *push* a **ECR** → `kubectl set image + rollout status` sobre **EKS**. Los secretos (credenciales AWS y contraseña de MySQL) viven en un **GitHub Environment**, nunca en el repositorio.

4. Desarrollo de la Solución

A continuación se desarrolla cada indicador de evaluación (IE1–IE7). Para cada uno se indica la exigencia de la rúbrica para alcanzar el 100% de logro, la implementación realizada y su justificación, y la evidencia gráfica con los comandos de CloudShell empleados.

IE1 · Configuración del Clúster en AWS (25%)

Exigencia de la rúbrica · 100% de logro (ponderación 25%)

"Clúster completamente funcional, roles IAM, VPC/subredes/SG correctos, nodos/capacity providers operativos, arquitectura clara."

Se creó la VPC **tienda-vpc** (10.0.0.0/16) con **6 subredes en 2 AZ** (2 públicas + 4 privadas) y un **NAT Gateway** para la salida a Internet de las subredes privadas. El clúster **tienda-eks** (Kubernetes 1.35) usa el rol IAM **LabRole** tanto para el plano de control como para los nodos, y un **node group** de instancias `t3.medium` (deseado/mín/máx = 2/2/4) ubicado en las **subredes privadas**.

Justificación del tipo de instancia (`t3.medium`) y descarte de `t2.micro/nano`

Se eligió **t3.medium** (2 vCPU, 4 GiB RAM) como mínimo viable, descartando instancias más pequeñas pese a su menor costo, por tres razones técnicas:

- **Límite de IPs por nodo (factor decisivo):** el plugin Amazon VPC CNI asigna una IP de la VPC a cada pod, y la cantidad disponible depende del tamaño de la instancia. `t2.micro/nano` permiten ~4 pods por nodo; `t3.medium` permite ~17. Con solo 4 IPs no alcanza ni para los pods de sistema.
- **Pods de sistema:** cada nodo ya ejecuta CoreDNS, kube-proxy, aws-node (CNI) y Metrics Server, lo que consume casi toda la capacidad de un `t2.micro` y dejaría a la app sin espacio ni margen para escalar (pods en *Pending*).
- **Memoria:** 0.5–1 GiB no basta para kubelet + pods de sistema + MySQL + Node/Express; los nodos quedarían *NotReady*. Los 4 GiB de `t3.medium` dan holgura para operar y escalar.

El ahorro de una instancia más pequeña es ilusorio, porque rompería la demo del autoscaling (HPA), que es el núcleo del indicador de escalabilidad. El control de créditos se gestionó por otras vías: *desired=2*, observabilidad apagada y *Stop Lab* al terminar cada sesión.

Permisos IAM (políticas incluidas en **LabRole**)

No fue necesario crear ni adjuntar políticas: AWS Academy entrega **LabRole** con un conjunto amplio de políticas administradas (y Learner Lab bloquea editar roles IAM).

Las 3 políticas clave para los nodos worker son:

- **AmazonEKSWorkerNodePolicy** — registra y opera la instancia EC2 como nodo del clúster.
- **AmazonEKS_CNI_Policy** — habilita el plugin de red (VPC CNI) para asignar IPs a los pods.
- **AmazonEC2ContainerRegistryReadOnly** — permiso de solo lectura sobre ECR (autoriza el *docker pull* de las imágenes).

El rol del plano de control no hace *pull* de imágenes; su política relevante es **AmazonEKSClusterPolicy**, también incluida en LabRole.

Justificación del diseño de red (patrón de producción)

Se evaluaron dos topologías. **Camino A (lab simplificado)**: nodos en subredes públicas con IP pública y sin NAT — más barato, pero expone los nodos y deja sin uso las subredes privadas. **Camino B (elegido, producción)**: nodos en subredes privadas (sin IP pública), un NAT Gateway para su salida y el Load Balancer en las públicas. Se eligió B por **seguridad** (el único punto de entrada es el ELB), **defensa en capas** (aunque un pod sea comprometido, el nodo no está expuesto), **coherencia** (las 6 subredes cumplen un rol real) y porque es la **arquitectura recomendada por AWS** para EKS. El costo del NAT se mitigó usando uno solo y apagando el lab al terminar.

Acceso remoto a los nodos = deshabilitado (sin SSH)

En el node group se dejó **OFF** la opción de acceso remoto: toda la administración se hace con `kubectl` desde CloudShell (que habla con el API del clúster, no con los nodos por SSH). Habilitar esa opción abriría el puerto 22 y contradiría el diseño de red privada. Si excepcionalmente se necesitara una shell en un nodo, se usaría **AWS SSM Session Manager** (no abre el puerto 22, funciona sobre nodos privados y queda auditado por IAM), de modo que dejar el acceso remoto en OFF no elimina ese plan B.

Evidencia

The screenshot displays the 'Crear VPC' (Create VPC) page in the AWS Management Console. The left sidebar shows the configuration details for the VPC 'tienda-vpc', including the CIDR block '10.0.0.0/16' and the option to generate automatic tags. The main area shows a preview of the VPC configuration, including the subnets and routing tables. The subnets are organized into two groups: 'us-east-1a' and 'us-east-1b', each containing two subnets (one public and one private). The routing tables are also shown, with one public and one private table for each availability zone.

Figura. VPC tienda-vpc creada con 6 subredes (2 públicas + 4 privadas) y NAT Gateway.

Ha cambiado correctamente la configuración de la subred:

- Habilitar la asignación automática de la dirección IPv4 pública

Subredes (6) Información

Buscar subredes por atributo o etiqueta

tienda X Quitar filtros

	Name	ID de subred	Estado	VPC	Bloquear el ...	CIDR IPv4	CIDR IPv6	ID de as...
☐	tienda-vpc-subnet-private2-us-east-1b	subnet-0dddb5fd1c1e6a94	Available	vpc-0063c9fa9aeef55c97 tienda...	☒ Desactivado	10.0.144.0/20	-	-
☐	tienda-vpc-subnet-public2-us-east-1b	subnet-08f89bc377809474	Available	vpc-0063c9fa9aeef55c97 tienda...	☒ Desactivado	10.0.16.0/20	-	-
☐	tienda-vpc-subnet-private3-us-east-1a	subnet-021d07e9e88ca5f0	Available	vpc-0063c9fa9aeef55c97 tienda...	☒ Desactivado	10.0.160.0/20	-	-
☐	tienda-vpc-subnet-public1-us-east-1a	subnet-0dc6982e522803d76	Available	vpc-0063c9fa9aeef55c97 tienda...	☒ Desactivado	10.0.0.0/20	-	-
☐	tienda-vpc-subnet-private1-us-east-1a	subnet-0e02bec35f503cf6	Available	vpc-0063c9fa9aeef55c97 tienda...	☒ Desactivado	10.0.128.0/20	-	-
☐	tienda-vpc-subnet-private4-us-east-1b	subnet-084d8cb0ba658d6fe	Available	vpc-0063c9fa9aeef55c97 tienda...	☒ Desactivado	10.0.176.0/20	-	-

Administrar las etiquetas para subnet-08f89bc3778094b74

Una etiqueta es una marca que se asigna a un recurso de AWS. Cada etiqueta consta de una clave y un valor opcional. Puede utilizar las etiquetas para buscar y filtrar sus recursos o hacer un seguimiento de los costos de AWS.

Valor - opcional

✕ Quitar

✕ Quitar

✕ Quitar

✕ Quitar

✕ Quitar

Agregar nueva etiqueta

Cancelar

Guardar

Figura. Etiqueta `kubernetes.io/role/elb = 1` en subred pública (descubrimiento del ELB público).

☰ VPC > Subredes > subnet-0e02bec35f503cfa6 > Administrar etiquetas

Administrar las etiquetas para subnet-0e02bec35f503cfa6

Una etiqueta es una marca que se asigna a un recurso de AWS. Cada etiqueta consta de una clave y un valor opcional. Puede utilizar las etiquetas para buscar y filtrar sus recursos o hacer un seguimiento de los costos de AWS.

Valor - opcional

✕ Quitar

✕ Quitar

✕ Quitar

✕ Quitar

✕ Quitar

Agregar nueva etiqueta

Cancelar

Guardar

Figura. Etiqueta `kubernetes.io/role/internal-elb = 1` en subred privada.

Acceso al punto de enlace del clúster [Información](#)

Configure el acceso al punto de enlace del servidor de la API de Kubernetes.

- ☐ **Público**
Se puede obtener acceso al punto de enlace del clúster desde fuera de la VPC. El tráfico del nodo de trabajo dejará la VPC para conectarse al punto de enlace.
- ☒ **Público y privado**
Se puede obtener acceso al punto de enlace del clúster desde fuera de la VPC. El tráfico del nodo de trabajo al punto de enlace permanecerá dentro de la VPC.
- ☐ **Privado**
Solo se puede obtener acceso al punto de enlace del clúster a través de la VPC. El tráfico desde el nodo de trabajo al punto de enlace permanecerá dentro de la VPC.

► **Configuración avanzada**

Salida del plano de control [Información](#)

Elige cómo se dirige el tráfico del plano de control de Kubernetes. En cuanto se cree un clúster, puede migrar de "Administrado por AWS" a "Enrutado por el cliente", pero no al revés.

Modo de salida | [Información](#)

- ☒ **Administrado por AWS**
El tráfico del plano de control sigue la ruta de red pública predeterminada.
- ☐ **Enrutado por el cliente**
El tráfico del plano de control se dirige a través de la VPC.

Cancelar

Anterior

Siguiente

Figura. Acceso al endpoint del clúster configurado como «Público y privado».

Paso 4

☒ **Seleccionar complementos**

Paso 5

☐ Configurar las opciones de complementos seleccionados

Paso 6

☐ Revisar y crear

☐

Agente de supervisión de nodos [Información](#)
Habilite la detección automática de problemas de estado de los nodos.
Categoría
observability
Computación compatible
EC2 y Nodos híbridos

☒

CoreDNS [Información](#)
Habilite la detección de servicios en el clúster.
Categoría
networking
Computación compatible
EC2, Nodos híbridos, Fargate, Modo automático de EKS y Clúster local

☒

kube-proxy [Información](#)
Habilite la red del servicio dentro del clúster.
Categoría
networking
Computación compatible
EC2, Nodos híbridos y Clúster local

☒

CNI de Amazon VPC [Información](#)
Habilite la red del pod dentro del clúster.
Categoría
networking
Computación compatible
EC2 y Clúster local

Figura. Complementos del clúster: CoreDNS, kube-proxy y CNI de Amazon VPC.

Revisar y crear

Paso 1: clúster

[Editar](#)

Configuración del clúster

Nombre
tienda-eks

Versión de Kubernetes
1.35

Modo automático de EKS
Desactivado

Política de actualización
Soporte estándar

Rol de IAM del clúster
arn:aws:iam::268450856324:role/LabRole

Acceso de administrador del clúster de Kubernetes
Permitir el acceso de administrador del clúster

Modo de autenticación
API de EKS

Nivel de escalado del plano de control
Estándar

Cambio de zona de ARC

Cambio de zona de ARC
Desactivado

Protección contra eliminaciones

Protección contra eliminaciones
Desactivado

Figura. Revisión del clúster: rol LabRole, modo de autenticación «API de EKS», Kubernetes 1.35.

Amazon Elastic Kubernetes Service

Clústeres > tienda-eks

Panel

Clústeres

Ajustes

- Configuración del panel
- Configuración de la consola

Amazon EKS Anywhere

- Suscripciones a Enterprise

Servicios relacionados

- Amazon ECR
- AWS Batch

Documentación

Talleres EKS

Novedad: ahora puede suscribirse a Información de contenedores de OTel con compatibilidad con OpenTelemetry y PromQL a partir de la versión 6.2.0 del complemento Observabilidad de CloudWatch. Más información

Notifications 0 0 0 4 0

tienda-eks

Conectar

Eliminar clúster

Actualizar versión

Clúster de monitores

El soporte estándar para la versión de Kubernetes 1.35 finaliza el 26 de marzo de 2027.

Actualizar

Información del clúster

Estado Activo	Versión de Kubernetes 1.35
Periodo de soporte Soporte estándar hasta el 26 de marzo de 2027	Proveedor EKS
Estado del clúster 0	Información sobre la actualización 4
Problemas de estado del nodo 0	Problemas de capacidad 0

Figura. Clúster tienda-eks en estado Active.

Informe EP3 · Tienda Perritos · AWS EKS

Página 10

Establecer la configuración informática y de escalado

Configuración de computación del grupo de nodos

Estas propiedades no se pueden cambiar después de crear el grupo de nodos.

Tipo de AMI [Información](#)

Seleccione la Amazon Machine Image optimizada para EKS para los nodos.

Amazon Linux 2023 (x86_64) Estándar (AL2023_x86_64_STANDARD) ▼

Tipo de capacidad

Seleccione la opción de compra de capacidad para este grupo de nodos.

On-Demand ▼

Tipos de instancias [Información](#)

Seleccione los tipos de instancia que prefiera para este grupo de nodos.

🔍 Ingrese un tipo de instancia

t3.medium

vCPU: 2 vCPUs Memory: 4 GiB Network: Up to 5 Gigabit Max ENI: 3 Max IPs: 18 ✕

Tamaño del disco

Seleccione el tamaño del volumen de EBS adjunto para cada nodo.

20

GiB

Figura. Node group tienda-nodes: AMI AL2023, tipo t3.medium, disco 20 GiB.

Configuración de escalado del grupo de nodos

Tamaño deseado

Establezca el número deseado de nodos con los que el grupo debería lanzarse inicialmente.

2

nodos

El tamaño del nodo deseado debe ser mayor o igual que 0

Tamaño mínimo

Establezca el número mínimo de nodos al que el grupo puede escalar de forma descendente.

2

nodos

El tamaño mínimo del nodo debe ser mayor o igual que 0

Tamaño máximo

Establezca el número máximo de nodos al que el grupo puede escalar de forma ascendente.

4

nodos

El tamaño máximo del nodo debe ser mayor o igual que 1 y no puede ser inferior al tamaño mínimo

Figura. Escalado del node group: deseado / mínimo / máximo = 2 / 2 / 4.

Paso 1

Configurar grupo de nodos

Paso 2

Establecer la configuración informática y de escalado

Paso 3

Especificar redes

Paso 4

Revisar y crear

Especificar redes

Configuración de red del grupo de nodos

Estas propiedades no se pueden cambiar después de crear el grupo de nodos.

Subredes [Información](#)

Especifique las subredes de la VPC donde se ejecutarán los nodos. Para crear una nueva subred, vaya a la página correspondiente en la [Consola de VPC](#).

Seleccionar subredes

subnet-0dddb5fd1c1e6a94 | tienda-vpc-subnet-private2-us-east-1b

us-east-1b 10.0.144.0/20

subnet-021d07e9e88ca45f0 | tienda-vpc-subnet-private3-us-east-1a

us-east-1a 10.0.160.0/20

subnet-0e02bec35f503cfa6 | tienda-vpc-subnet-private1-us-east-1a

us-east-1a 10.0.128.0/20

subnet-084d8c0ba4658d66e | tienda-vpc-subnet-private4-us-east-1b

us-east-1b 10.0.176.0/20

Borrar subredes seleccionadas

☐ Configurar el acceso remoto a los nodos [Información](#)

Cancelar

Anterior

Siguiente

Figura. Node group asociado a las 4 subredes privadas; acceso remoto (SSH) en OFF.

CloudShell

us-east-1 x us-east-1 x us-east-1 x +

```
~ $ kubectl get nodes -o wide -L topology.kubernetes.io/zone
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME	ZONE
ip-10-0-131-186.ec2.internal	Ready	<none>	97m	v1.35.6-eks-93b88c6	10.0.131.186	<none>	Amazon Linux 2023.12.20268611	6.12.90-120.164.amzn2023.x86_64	containerd://2.2.4+unknown	us-east-1a
ip-10-0-181-187.ec2.internal	Ready	<none>	99m	v1.35.6-eks-93b88c6	10.0.181.187	<none>	Amazon Linux 2023.12.20268611	6.12.90-120.164.amzn2023.x86_64	containerd://2.2.4+unknown	us-east-1b

Figura. Dos nodos en estado Ready, uno por AZ (us-east-1a / us-east-1b), con EXTERNAL-IP = sin IP pública.

```
kubectl get nodes -o wide -L topology.kubernetes.io/zone
```

IE2 · Despliegue Frontend + Backend (25%)

Exigencia de la rúbrica · 100% de logro (ponderación 25%)

"Servicios desplegados correctamente desde ECR, variables correctas, puertos y balanceador operativo."

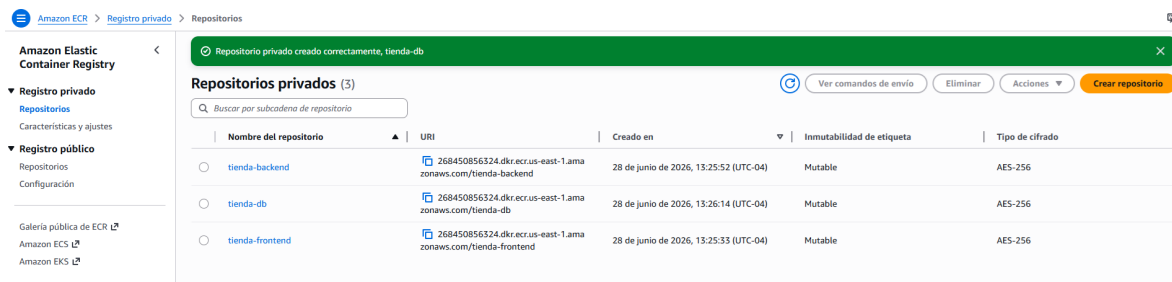
Se crearon tres repositorios privados en **Amazon ECR** (tienda-frontend, tienda-backend, tienda-db). Las imágenes se despliegan desde ECR con sus **variables de entorno** cargadas (DB_HOST, DB_USER, etc.). El balanceo se resuelve con el Service tienda-frontend de tipo LoadBalancer, que aprovisiona un **Classic ELB** público. La comunicación Front→Back→DB usa el **DNS interno** de Kubernetes.

URL pública (EXTERNAL-IP del ELB):

<http://a49d9bbc548ab48b19dddc7f331bf7c4-1672428966.us-east-1.elb.amazonaws.com> (HTTP, puerto 80).

El EXTERNAL-IP se verificó por dos vías equivalentes: (1) automatizada, en los logs del pipeline; y (2) manual, desde CloudShell tras `aws eks update-kubeconfig`.

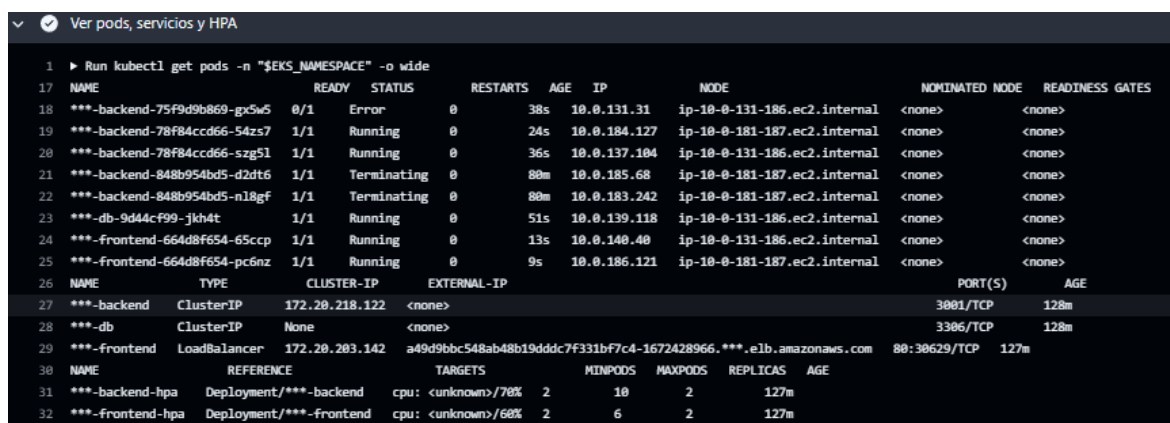
Evidencia



The screenshot shows the Amazon ECR console interface. On the left, there's a navigation menu with 'Registro privado' and 'Registro público'. The main area displays a list of private repositories under the heading 'Repositorios privados (3)'. The list includes three repositories: 'tienda-backend', 'tienda-db', and 'tienda-frontend'. Each repository entry shows its URI, creation date, mutability, and encryption type (AES-256).

Nombre del repositorio	URI	Creado en	Inmutabilidad de etiqueta	Tipo de cifrado
tienda-backend	268450856324.dkr.ecr.us-east-1.amazonaws.com/tienda-backend	28 de junio de 2026, 13:25:52 (UTC-04)	Mutable	AES-256
tienda-db	268450856324.dkr.ecr.us-east-1.amazonaws.com/tienda-db	28 de junio de 2026, 13:26:14 (UTC-04)	Mutable	AES-256
tienda-frontend	268450856324.dkr.ecr.us-east-1.amazonaws.com/tienda-frontend	28 de junio de 2026, 13:25:33 (UTC-04)	Mutable	AES-256

Figura. Los 3 repositorios privados en Amazon ECR (frontend, backend y db).



The screenshot shows a terminal window with the command `Run kubectl get pods -n "$EKS_NAMESPACE" -o wide` and its output. The output lists several pods in various states (Error, Running, Terminating) and a service named 'tienda-frontend' of type 'LoadBalancer'. The service's external IP is visible as 'a49d9bbc548ab48b19dddc7f331bf7c4-1672428966.us-east-1.elb.amazonaws.com'.

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS GATES
***-backend-75f9d9b869-gx5w5	0/1	Error	0	38s	10.0.131.31	ip-10-0-131-186.ec2.internal	<none>	<none>	<none>
***-backend-78f84ccd66-54zs7	1/1	Running	0	24s	10.0.184.127	ip-10-0-181-187.ec2.internal	<none>	<none>	<none>
***-backend-78f84ccd66-szg5l	1/1	Running	0	36s	10.0.137.104	ip-10-0-131-186.ec2.internal	<none>	<none>	<none>
***-backend-848b954bd5-d2dt6	1/1	Terminating	0	80m	10.0.185.68	ip-10-0-181-187.ec2.internal	<none>	<none>	<none>
***-backend-848b954bd5-nl8gf	1/1	Terminating	0	80m	10.0.183.242	ip-10-0-181-187.ec2.internal	<none>	<none>	<none>
***-db-9d44cf99-jkh4t	1/1	Running	0	51s	10.0.139.118	ip-10-0-131-186.ec2.internal	<none>	<none>	<none>
***-frontend-664d8f654-65ccp	1/1	Running	0	13s	10.0.140.40	ip-10-0-131-186.ec2.internal	<none>	<none>	<none>
***-frontend-664d8f654-pc6nz	1/1	Running	0	9s	10.0.186.121	ip-10-0-181-187.ec2.internal	<none>	<none>	<none>

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
***-backend	ClusterIP	172.20.218.122	<none>	3001/TCP	128m
***-db	ClusterIP	None	<none>	3306/TCP	128m
***-frontend	LoadBalancer	172.20.203.142	a49d9bbc548ab48b19dddc7f331bf7c4-1672428966.us-east-1.elb.amazonaws.com	80:30629/TCP	127m

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
-backend-hpa	Deployment/-backend	cpu: <unknown>/70%	2	10	2	127m
-frontend-hpa	Deployment/-frontend	cpu: <unknown>/60%	2	6	2	127m

Figura. EXTERNAL-IP del ELB visible en los logs de GitHub Actions.

```
aws CloudShell
us-east-1 +
~ $ aws eks update-kubeconfig --region us-east-1 --name tienda-eks
Added new context arn:aws:eks:us-east-1:268450856324:cluster/tienda-eks to /home/cloudshell-user/.kube/config
~ $ kubectl get svc -n tienda
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
tienda-backend      ClusterIP   172.20.218.122  <none>           3001/TCP         10m
tienda-db            ClusterIP   None            <none>           3306/TCP         10m
tienda-frontend     LoadBalancer 172.20.203.142  a49d9bbc548ab48b19dddc7f331bf7c4-1672428966.us-east-1.elb.amazonaws.com 80:30629/TCP     10m
~ $
```

Figura. Mismo EXTERNAL-IP obtenido desde CloudShell.

```
aws eks update-kubeconfig --region us-east-1 --name tienda-eks
kubectl get svc -n tienda
```

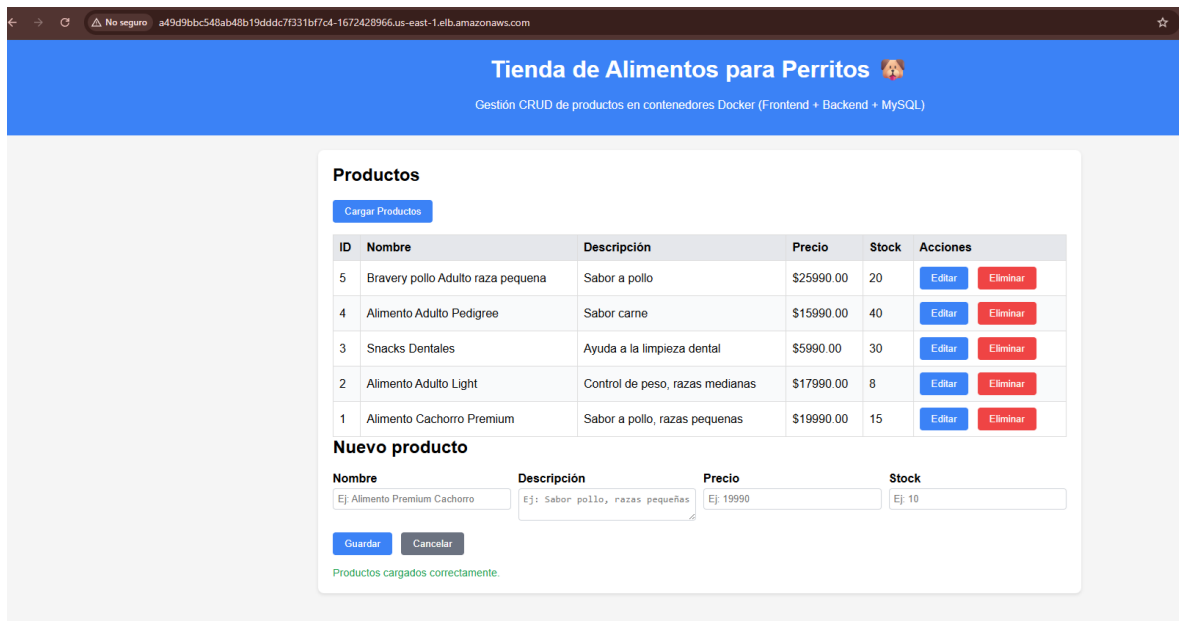


Figura. Tienda Perritos cargada en el navegador desde la URL pública del ELB.

IE3 · Configuración de Autoscaling — HPA (10%)

Exigencia de la rúbrica · 100% de logro (ponderación 10%)

"Autoscaling configurado correctamente, métricas funcionando y justificación del umbral."

Se configuró **HPA** para ambos servicios visibles: **backend** 2→10 réplicas con umbral **70% CPU**, y **frontend** 2→6 réplicas con umbral **60% CPU**. El requisito previo es el **Metrics Server**: se usa el **complemento gestionado del clúster EKS** (no el manifiesto de upstream). El HPA calcula el % de CPU contra el *request* de cada Deployment (backend request=100m / límite=500m), por lo que el umbral de 70% equivale a 70m de CPU.

Justificación del umbral (70% backend / 60% frontend)

Se deja margen antes de saturar el pod para que el HPA tenga tiempo de crear réplicas nuevas (que tardan en arrancar) **antes** de degradar el servicio. El frontend usa un umbral algo menor (60%) por ser la cara visible al usuario: conviene anticiparse más a los picos de tráfico.

Incidente resuelto: HPA en «<unknown>» / Metrics API caída

Síntoma: tras desplegar, el HPA mostraba `cpu <unknown>/70%` y `kubectl top pods` devolvía «Metrics API not available». **Diagnóstico:** el pipeline instalaba además el metrics-server de upstream, cuyo Service usa el selector `k8s-app`; al hacer `apply` sobre el Service ya creado por el complemento del EKS (etiqueta `app.kubernetes.io/name`), los selectores se fusionaron en un AND imposible → sin endpoints → Metrics API caída. **Solución de raíz:** se eliminó del pipeline la instalación de upstream y se dejó solo el complemento gestionado, añadiendo una verificación de la Metrics API.

Prueba de carga y escalado

Se generó carga artificial sobre el backend con un pod *load-generator* (10 bucles en paralelo de peticiones a `/api/health`) y se observó el HPA en vivo. Resultado: el CPU subió a 435%, el HPA escaló **2 → 4 → 8 → 10** réplicas (tope configurado) y, al repartirse la carga, el CPU promedio volvió bajo el umbral; al retirar la carga, volvió a 2 réplicas tras el enfriamiento.

Evidencia


```

~ $ kubectl get apiservices v1beta1.metrics.k8s.io
NAME                SERVICE                AVAILABLE  AGE
v1beta1.metrics.k8s.io  kube-system/metrics-server  True      107m
~ $ kubectl top pods -n tienda
NAME                CPU(cores)  MEMORY(bytes)
tienda-backend-848b954bd5-9ccrb  1m          21Mi
tienda-backend-848b954bd5-s2mzp  1m          21Mi
tienda-db-654f8b9d68-r9p5l      7m          423Mi
tienda-frontend-78df94cf8c-p8b2s  1m          3Mi
tienda-frontend-78df94cf8c-x6gqv  1m          3Mi
~ $ kubectl get hpa -n tienda
NAME                REFERENCE                TARGETS  MINPODS  MAXPODS  REPLICAS  AGE
tienda-backend-hpa  Deployment/tienda-backend  cpu: 1%/70%  2        10       2         16m
tienda-frontend-hpa  Deployment/tienda-frontend  cpu: 2%/60%  2        6        2         16m
~ $

```

Figura. Metrics API disponible: HPA con % real de CPU y kubectl top pods funcionando.

```

kubectl get apiservices v1beta1.metrics.k8s.io
kubectl top pods -n tienda
kubectl get hpa -n tienda

```

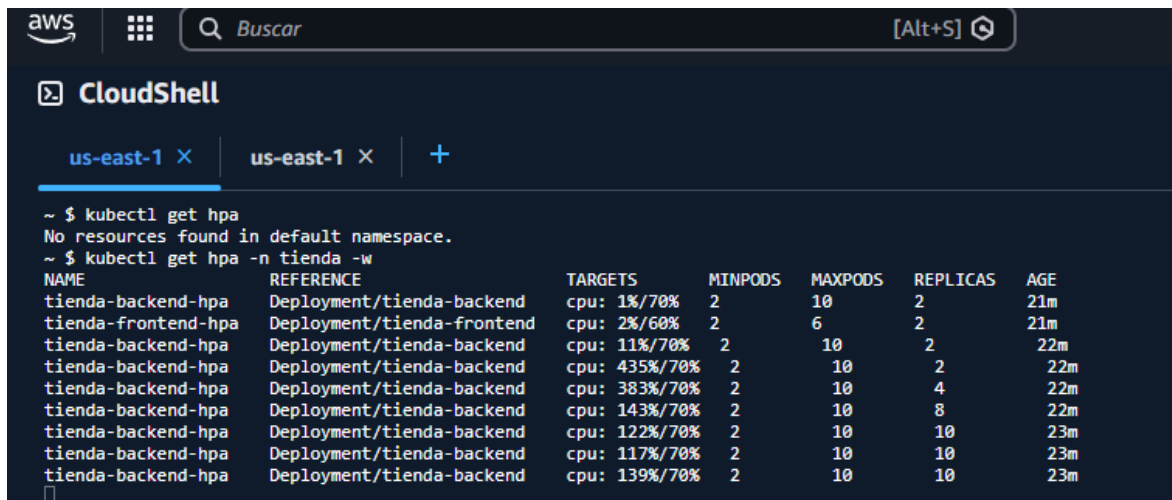


Figura. HPA escalando: CPU por sobre el umbral y número de réplicas subiendo (2→4→8).

```

kubectl get hpa -n tienda -w # con la carga activa

```

```
CloudShell
us-east-1 X us-east-1 X +
~ $ kubectl run -n tienda load-generator --image=busybox:1.28 --restart=Never -- \
> /bin/sh -c "for i in 1 2 3 4 5 6 7 8 9 10; do (while true; do \
> wget -q -O- http://tienda-backend:3001/api/health >/dev/null 2>&1; done) & done; wait"
pod/load-generator created
~ $
```

CloudShell

us-east-1 X us-east-1 X +

Every 2.0s: kubectl get hpa,pods -n tienda -o wide

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/tienda-backend-hpa	Deployment/tienda-backend	cpu: 1%/70%	2	10	2	143m
horizontalpodautoscaler.autoscaling/tienda-frontend-hpa	Deployment/tienda-frontend	cpu: 2%/60%	2	6	2	143m

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
pod/load-generator	1/1	Running	0	2s	10.0.138.13	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-54zs7	1/1	Running	0	16m	10.0.184.127	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-szg5l	1/1	Running	0	16m	10.0.137.104	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-db-9d44cf99-jkh4t	1/1	Running	0	16m	10.0.139.118	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-frontend-664d8f654-65ccp	1/1	Running	0	15m	10.0.140.40	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-frontend-664d8f654-pc6nz	1/1	Running	0	15m	10.0.186.121	ip-10-0-181-187.ec2.internal	<none>	<none>

CloudShell

us-east-1 X us-east-1 X +

Every 2.0s: kubectl get hpa,pods -n tienda -o wide

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/tienda-backend-hpa	Deployment/tienda-backend	cpu: 77%/70%	2	10	10	144m
horizontalpodautoscaler.autoscaling/tienda-frontend-hpa	Deployment/tienda-frontend	cpu: 2%/60%	2	6	2	144m

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
pod/load-generator	1/1	Running	0	69s	10.0.138.13	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-4bkbw	1/1	Running	0	47s	10.0.133.178	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-54zs7	1/1	Running	0	17m	10.0.184.127	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-5dtb8	1/1	Running	0	17s	10.0.181.134	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-6z6vt	1/1	Running	0	17s	10.0.133.92	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-b558g	1/1	Running	0	32s	10.0.183.242	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-dsvbc	1/1	Running	0	17s	10.0.185.68	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-gvsln	1/1	Running	0	17s	10.0.140.157	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-svv2p	1/1	Running	0	32s	10.0.133.70	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-szg5l	1/1	Running	0	17m	10.0.137.104	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-txxzc	1/1	Running	0	32s	10.0.186.248	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-db-9d44cf99-jkh4t	1/1	Running	0	17m	10.0.139.118	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-frontend-664d8f654-65ccp	1/1	Running	0	17m	10.0.140.40	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-frontend-664d8f654-pc6nz	1/1	Running	0	16m	10.0.186.121	ip-10-0-181-187.ec2.internal	<none>	<none>

Figura. Backend en carga (~77%/70%) con ~10 pods Running repartidos en ambos nodos.

```
# terminal de carga:
kubectl run -n tienda load-generator --image=busybox:1.28 --restart=Never -- \
/bin/sh -c "for i in 1 2 3 4 5 6 7 8 9 10; do (while true; do \
wget -q -O- http://tienda-backend:3001/api/health >/dev/null 2>&1; done) & done; wait"
# monitor:
watch -n 2 "kubectl get hpa,pods -n tienda -o wide"
```

CloudShell

us-east-1 X us-east-1 X us-east-1 X us-east-1 X +

```
~ $ kubectl delete pod load-generator -n tienda
pod "load-generator" deleted from tienda namespace
~ $ kubectl get hpa -n tienda -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
tienda-backend-hpa	Deployment/tienda-backend	cpu: 1%/70%	2	10	2	154m
tienda-frontend-hpa	Deployment/tienda-frontend	cpu: 2%/60%	2	6	2	154m

Figura. Scale-down: tras retirar la carga el HPA reduce las réplicas de vuelta a 2.

```
kubectl delete pod load-generator -n tienda
kubectl get hpa -n tienda -w # esperar ~5 min: REPLICAS -> 2
```

Mejora futura: un *Cluster Autoscaler* escalaría también los nodos EC2 cuando el HPA pida más pods de los que caben. No se implementó (limitación del Learner Lab), pero

queda documentado.

IE4 · Pipeline CI/CD — build → push → deploy (15%)

Exigencia de la rúbrica · 100% de logro (ponderación 15%)

"Pipeline completamente automatizado, funcional, seguro y documentado; deploy exitoso."

El workflow `.github/workflows/deploy-eks.yml` se dispara con cada *push* a **main** (y manualmente). Sus etapas son: configurar credenciales AWS → login en ECR → *build & push* de las tres imágenes (frontend, backend, db) → despliegue en EKS (`kubectl set image + rollout status`). El despliegue tras commit es **automático** y la corrida completa toma **~2 minutos** (corrida #2: 1m56s). Las imágenes se etiquetan con el SHA corto + latest para trazabilidad.

Evidencia

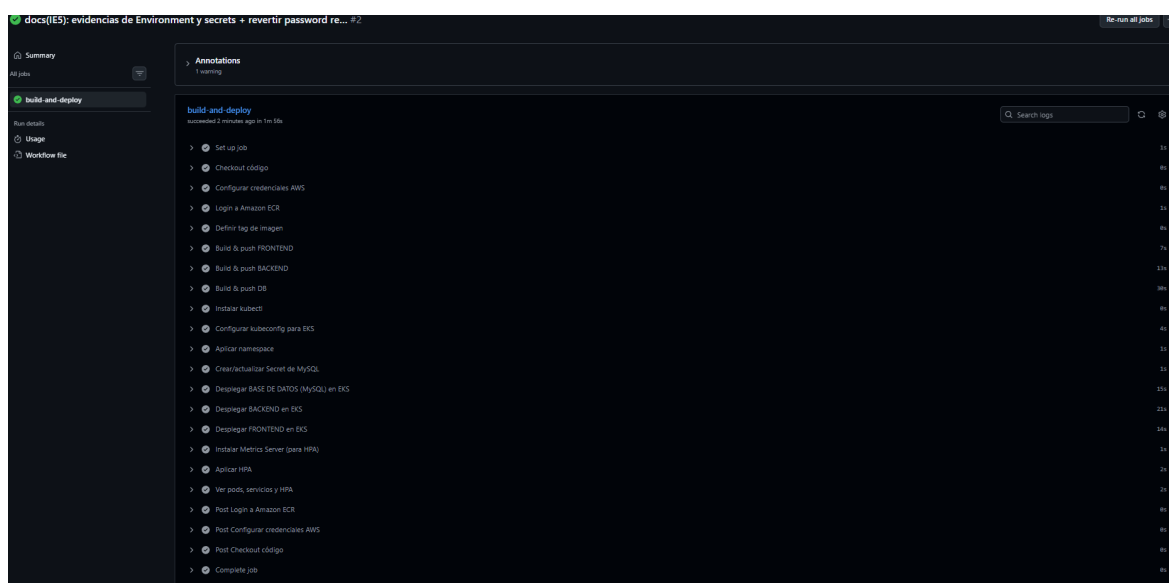


Figura. Pipeline de GitHub Actions en verde: build, push y deploy completados con éxito.

IE5 · Gestión de Secrets y Credenciales (5%)

Exigencia de la rúbrica · 100% de logro (ponderación 5%)

"Secrets correctamente utilizados, seguros y bien estructurados; sin exposición."

Los secretos viven en un **GitHub Environment «production»** (credenciales AWS, datos del clúster y `MYSQL_ROOT_PASSWORD`). Las credenciales de BD se manejan vía `.env` (en `.gitignore`) y el **Secret de Kubernetes se genera dentro del pipeline** a partir de los GitHub Secrets — nunca se versiona una contraseña. Se eligió un *Environment* (en lugar de secrets sueltos) por su **aislamiento de alcance**, sus **reglas de protección** y su **trazabilidad**.

Evidencia

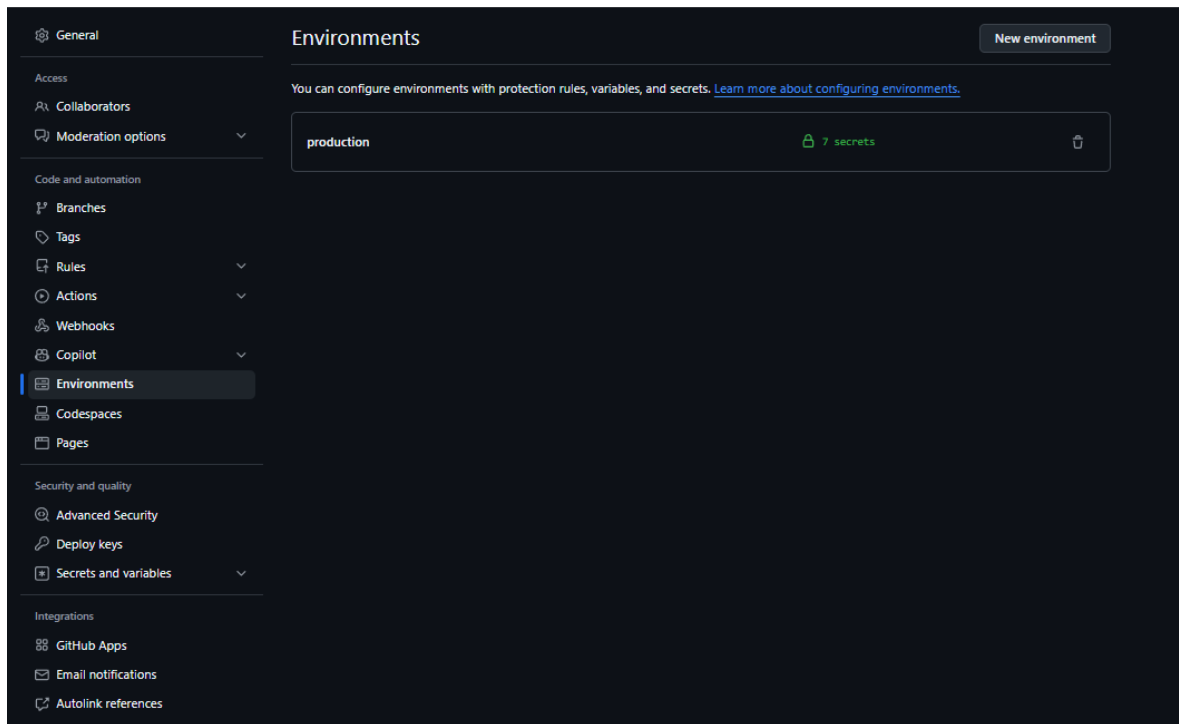


Figura. GitHub Environment «production» creado para aislar los secretos del despliegue.

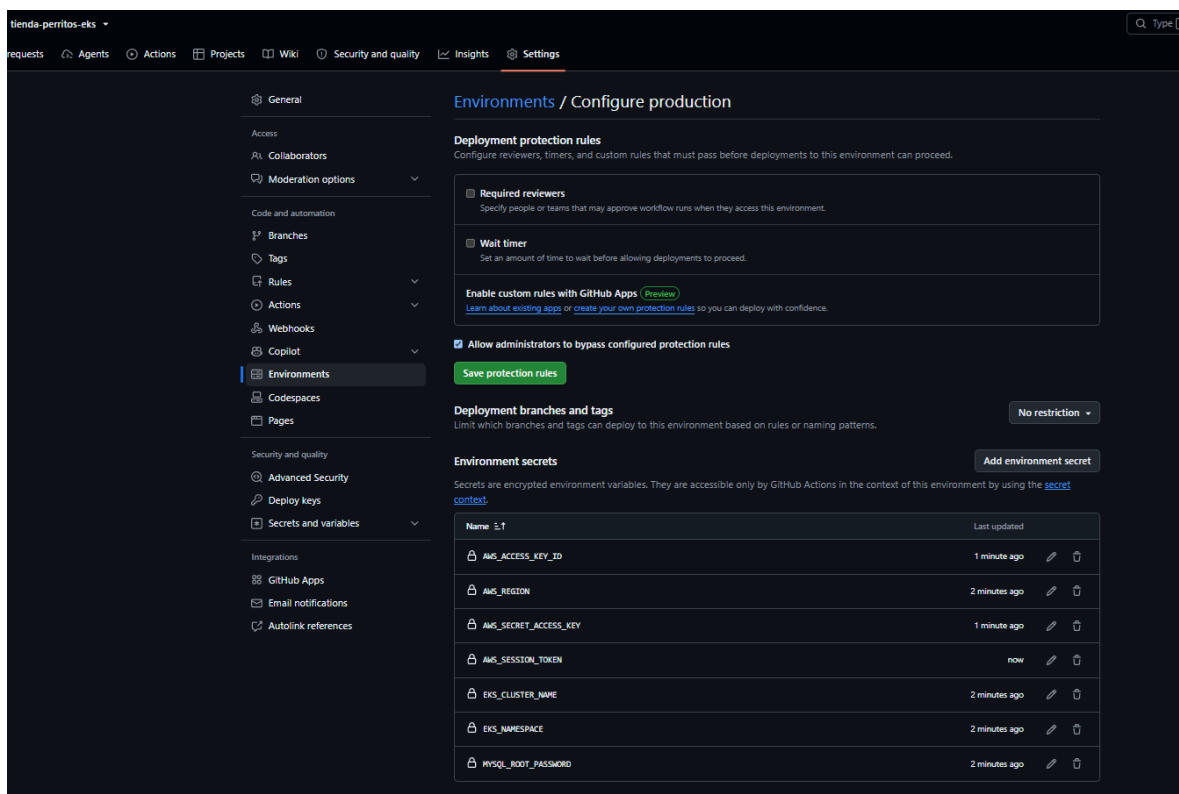


Figura. Secrets configurados en el Environment (AWS_*, EKS_*, MYSQL_ROOT_PASSWORD).

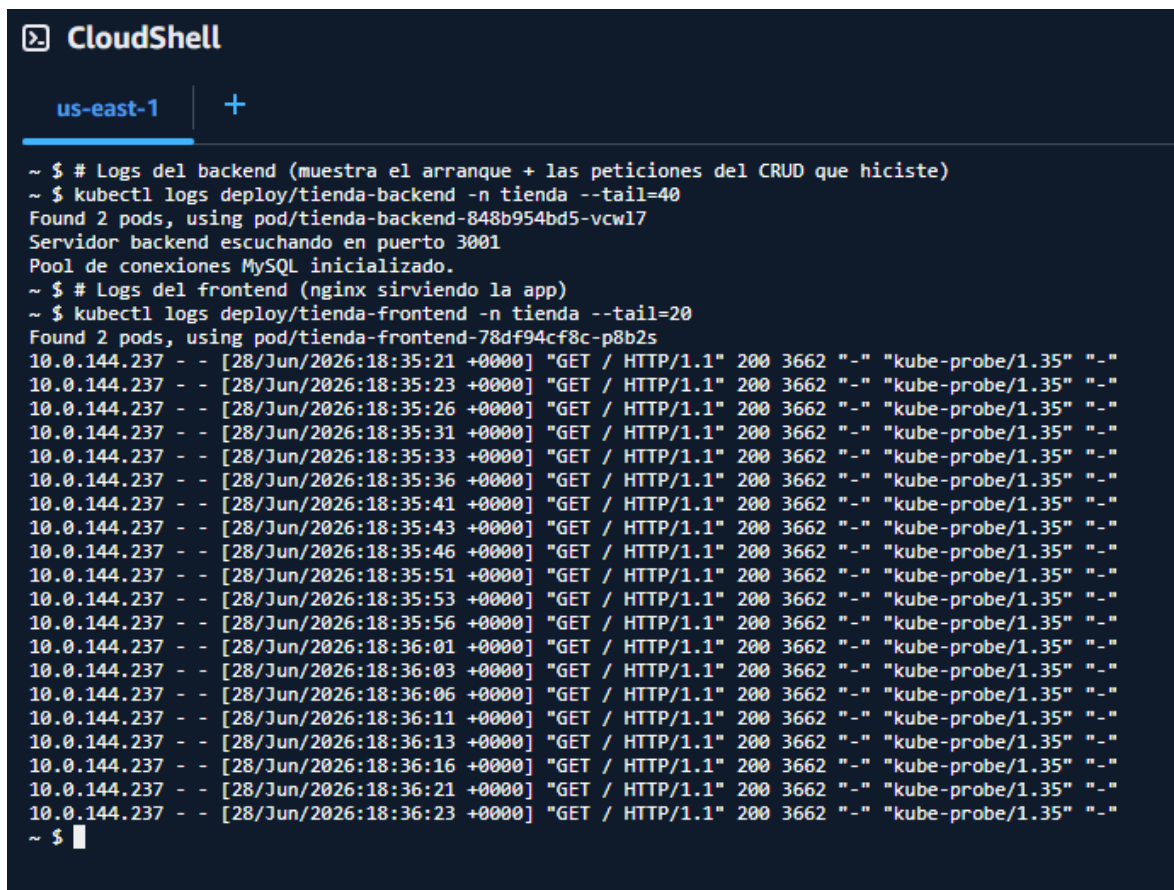
IE6 · Análisis de Logs, Métricas y Tiempos (10%)

Exigencia de la rúbrica · 100% de logro (ponderación 10%)

"Análisis completo de logs, errores, tiempos y métricas; conclusiones claras."

Logs: `kubectl logs` del backend muestra «Servidor backend escuchando en puerto 3001» y «Pool de conexiones MySQL inicializado» (conexión Back↔DB correcta); el frontend (nginx) registra respuestas **200 OK**, incluidas las sondas *kube-probe* (readiness/liveness funcionando). **Métricas:** uso de CPU/memoria por pod vía Metrics Server (`kubectl top pods`). **CloudWatch:** se activaron temporalmente los logs del plano de control (API Server + Auditoría) → log group /aws/eks/tienda-eks/cluster, desactivados tras la captura para no consumir créditos. **Tiempos:** ~2 min por corrida completa del pipeline.

Evidencia



```
CloudShell
us-east-1 +
~ $ # Logs del backend (muestra el arranque + las peticiones del CRUD que hiciste)
~ $ kubectl logs deploy/tienda-backend -n tienda --tail=40
Found 2 pods, using pod/tienda-backend-848b954bd5-vcwl7
Servidor backend escuchando en puerto 3001
Pool de conexiones MySQL inicializado.
~ $ # Logs del frontend (nginx sirviendo la app)
~ $ kubectl logs deploy/tienda-frontend -n tienda --tail=20
Found 2 pods, using pod/tienda-frontend-78df94cf8c-p8b2s
10.0.144.237 - - [28/Jun/2026:18:35:21 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:23 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:26 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:31 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:33 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:36 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:41 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:43 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:46 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:51 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:53 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:56 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:01 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:03 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:06 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:11 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:13 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:16 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:21 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:23 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
~ $
```

Figura. Logs de los pods: backend escuchando y pool MySQL OK; frontend respondiendo 200 OK.

```
kubectl logs deploy/tienda-backend -n tienda --tail=40
kubectl logs deploy/tienda-frontend -n tienda --tail=20
```

Amazon Elastic Kubernetes Service > Clusters > tienda-eks > Administrar registros

Administrar registro: tienda-eks

Registros del plano de control Submarinador
Envíe registros de auditoría y diagnóstico desde el plano de control de Amazon EKS a CloudWatch Logs.
☒ **Servidor de la API**
Registros que pertenecen a las solicitudes de la API al clúster.
☐ **Auditoría**
Registros que pertenecen al acceso al clúster a través de la API de Kubernetes.
☐ **Autenticador**
Registros que pertenecen a las solicitudes de autenticación en el clúster.
☐ **Administrador de controladores**
Registros que pertenecen al estado de los controladores del clúster.
☐ **Programador**
Registros que pertenecen a las decisiones de programación.

Cancelar Guardar cambios

/aws/eks/tienda-eks/cluster

Acciones

▼ Detalles del grupo de registros

Clase de registro Información
Estándar

ARN
arn:aws:logsus-east-1:268450856324:log-group:/aws/eks/tienda-eks/cluster:*

Hora de creación
-

Retención
No vence nunca

Bytes almacenados
-

Cuenta
-

Filtros de métricas

Filtros de suscripción
No compatible

Reglas de Contributor Insights
No compatible

ID de clave de KMS
-

Protección contra eliminación
Desactivado

Protección de datos
-

< Flujos de registros Etiquetas Protección de datos Detección de anomalías Filtros de métricas Filtros de s

Flujos de registros (2)

De forma predeterminada, solo se cargan los flujos de registro más recientes.

☐ Coincidencia exacta ☐ Mostrar caducado

☐ Secuencia de registro

▼ Hora del último evento

kube-apiserver-f10963a5086765a79bbbaa5b71ec24732026-06-28 18:39:26 (UTC)

kube-apiserver-073dff1727960122dbacfd0a3dff0f52026-06-28 18:36:44 (UTC)

Figura. CloudWatch: log group /aws/eks/tienda-eks/cluster con los streams del API Server.

Informe EP3 · Tienda Perritos · AWS EKS

Página 22

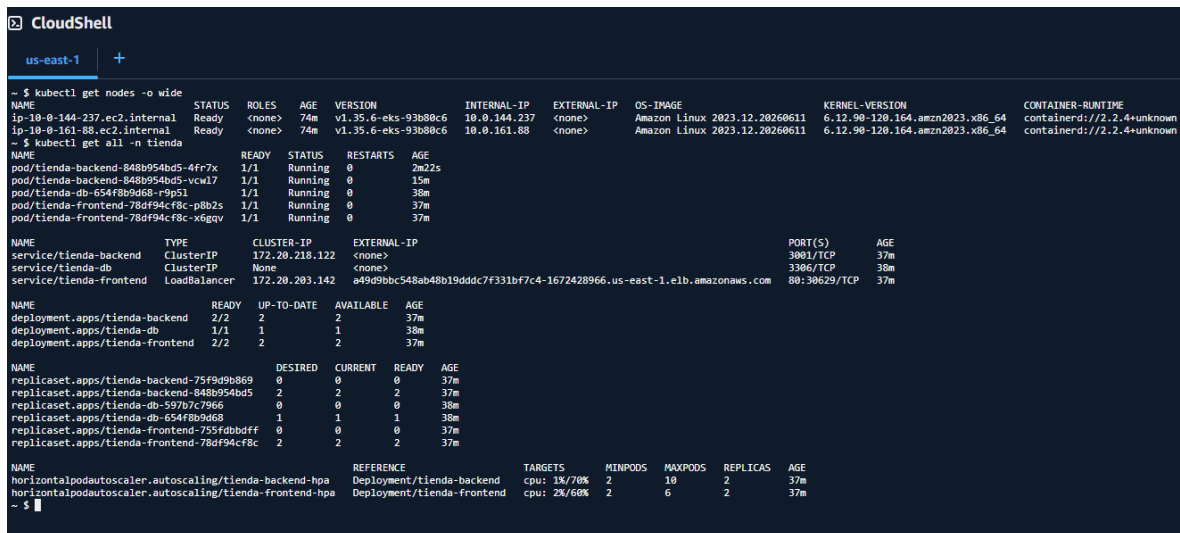
IE7 · Validación Funcional del Clúster (10%)

Exigencia de la rúbrica · 100% de logro (ponderación 10%)

"Servicios operativos, comunicación correcta, endpoints funcionando, logs y recuperación post-deploy demostrados."

Se validó el funcionamiento integral de Tienda Perritos: **CRUD completo** de productos desde la web (crear / editar / eliminar) con la comunicación Front→Back→DB operativa. Se comprobó la **autorrecuperación** de Kubernetes: al borrar un pod del backend, el clúster creó automáticamente uno nuevo (AGE 15s) manteniendo las 2 réplicas. La **recuperación post-redeploy** también se verificó: cada *push* ejecuta *set image + rollout status*, reemplazando los pods sin caída del servicio.

Evidencia



```
~ $ kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                                     KERNEL-VERSION   CONTAINER-RUNTIME
ip-10-0-144-237.ec2.internal        Ready    <none>    74m   v1.35.6-eks-93b88c6   10.0.144.237   <none>         Amazon Linux 2023.12.20260611             6.12.90-120.164.amzn2023.x86_64   containerd://2.2.4+unknown
ip-10-0-161-88.ec2.internal         Ready    <none>    74m   v1.35.6-eks-93b88c6   10.0.161.88    <none>         Amazon Linux 2023.12.20260611             6.12.90-120.164.amzn2023.x86_64   containerd://2.2.4+unknown

~ $ kubectl get all -n tienda
NAME                                READY    STATUS    RESTARTS   AGE
pod/tienda-backend-848b954bd5-4fp7x 1/1      Running   0           2m22s
pod/tienda-backend-848b954bd5-vcw17 1/1      Running   0           15m
pod/tienda-db-654f8b9d68-r9p5l       1/1      Running   0           38m
pod/tienda-frontend-78df94cf8c-p8b2s 1/1      Running   0           37m
pod/tienda-frontend-78df94cf8c-x6gqv 1/1      Running   0           37m

NAME                                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
service/tienda-backend              ClusterIP    172.20.218.122 <none>         3001/TCP          37m
service/tienda-db                    ClusterIP    None          <none>         3306/TCP          38m
service/tienda-frontend              LoadBalancer 172.20.203.142 a49d9bbc548ab48b19dddc7f331bf7c4-1672428966.us-east-1.elb.amazonaws.com 80:30629/TCP 37m

NAME                                READY    UP-TO-DATE   AVAILABLE   AGE
deployment.apps/tienda-backend      2/2      2             2           37m
deployment.apps/tienda-db           1/1      1             1           38m
deployment.apps/tienda-frontend      2/2      2             2           37m

NAME                                DESIRED    CURRENT   READY   AGE
replicaset.apps/tienda-backend-75f9db869 0          0         0       37m
replicaset.apps/tienda-backend-848b954bd5 2          2         2       37m
replicaset.apps/tienda-db-597b7c7966      0          0         0       38m
replicaset.apps/tienda-db-654f8b9d68      1          1         1       38m
replicaset.apps/tienda-frontend-755fdbbdf 0          0         0       37m
replicaset.apps/tienda-frontend-78df94cf8c 2          2         2       37m

NAME                                REFERENCE                                     TARGETS  MINPODS  MAXPODS  REPLICAS  AGE
horizontalpodautoscaler.autoscaling/tienda-backend-hpa Deployment/tienda-backend                    cpu: 1X/70%  2         10         2         37m
horizontalpodautoscaler.autoscaling/tienda-frontend-hpa Deployment/tienda-frontend                    cpu: 2X/60%  2          6         2         37m
```

Figura. Estado sano del clúster: 2 nodos Ready y todos los recursos del namespace tienda.

```
kubectl get nodes -o wide
kubectl get all -n tienda
```



```
us-east-1 X us-east-1 X +
~ $ kubectl get pods -n tienda
NAME                                READY   STATUS    RESTARTS   AGE
tienda-backend-848b954bd5-9ccrb    1/1     Running   0           34m
tienda-backend-848b954bd5-vcwl7    1/1     Running   0           11m
tienda-db-654f8b9d68-r9p5l         1/1     Running   0           34m
tienda-frontend-78df94cf8c-p8b2s   1/1     Running   0           34m
tienda-frontend-78df94cf8c-x6gqv   1/1     Running   0           34m
~ $ kubectl delete pod tienda-backend-848b954bd5-9ccrb -n tienda
pod "tienda-backend-848b954bd5-9ccrb" deleted from tienda namespace
```

```
~ $ kubectl get pods -n tienda
NAME                                READY   STATUS    RESTARTS   AGE
tienda-backend-848b954bd5-4fr7x    1/1     Running   0           15s
tienda-backend-848b954bd5-9ccrb    1/1     Terminating   0           35m
tienda-backend-848b954bd5-vcwl7    1/1     Running   0           12m
tienda-db-654f8b9d68-r9p5l         1/1     Running   0           36m
tienda-frontend-78df94cf8c-p8b2s   1/1     Running   0           35m
tienda-frontend-78df94cf8c-x6gqv   1/1     Running   0           35m
~ $
```

Figura. Autorrecuperación: al eliminar un pod del backend, Kubernetes recrea otro automáticamente.

```
kubectl get pods -n tienda
kubectl delete pod <pod-tienda-backend> -n tienda
kubectl get pods -n tienda # aparece el nuevo pod recreado
```

Tienda de Alimentos para Perritos 🐶

Gestión CRUD de productos en contenedores Docker (Frontend + Backend + MySQL)

Productos

Cargar Productos

ID	Nombre	Descripción	Precio	Stock	Acciones
6	NUEVO PRODUCTO PÁPI	POLLITO	\$1313.00	2	<button>Editar</button> <button>Eliminar</button>
5	Bravery pollo Adulto raza pequena	Sabor a pollo	\$25990.00	23	<button>Editar</button> <button>Eliminar</button>
4	Alimento Adulto Pedigree	Sabor carne	\$15990.00	40	<button>Editar</button> <button>Eliminar</button>
3	Snacks Dentales	Ayuda a la limpieza dental	\$5990.00	30	<button>Editar</button> <button>Eliminar</button>

Nuevo producto

Nombre

Ej: Alimento Premium Cachorro

Descripción

Ej: Sabor pollo, razas pequeñas

Precio

Ej: 19990

Stock

Ej: 10

Guardar Cancelar

Producto creado correctamente.

Figura. CRUD funcional en la web: alta/edición/baja de productos (comunicación Front→Back→DB).

```

~ $ kubectl get pods -n tienda -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
tienda-backend-848b954bd5-d2dt6    1/1     Running   0           74m   10.0.185.68     ip-10-0-181-187.ec2.internal       <none>            <none>
tienda-backend-848b954bd5-nl8gf    1/1     Running   0           74m   10.0.183.242    ip-10-0-181-187.ec2.internal       <none>            <none>
tienda-db-654f8b9d68-gndv2         1/1     Running   0           74m   10.0.176.23     ip-10-0-181-187.ec2.internal       <none>            <none>
tienda-frontend-78df94cf8c-7vkkt    1/1     Running   0           74m   10.0.186.248    ip-10-0-181-187.ec2.internal       <none>            <none>
tienda-frontend-78df94cf8c-hgbf8    1/1     Running   0           74m   10.0.181.134    ip-10-0-181-187.ec2.internal       <none>            <none>
~ $ kubectl get pods -n tienda -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
tienda-backend-78f84ccd66-54zs7    1/1     Running   0           48s   10.0.184.127    ip-10-0-181-187.ec2.internal       <none>            <none>
tienda-backend-78f84ccd66-szg5l    1/1     Running   0           60s   10.0.137.104    ip-10-0-131-186.ec2.internal       <none>            <none>
tienda-db-9d44cf99-jkh4t           1/1     Running   0           75s   10.0.139.118    ip-10-0-131-186.ec2.internal       <none>            <none>
tienda-frontend-664d8f654-65ccp     1/1     Running   0           37s   10.0.140.40     ip-10-0-131-186.ec2.internal       <none>            <none>
tienda-frontend-664d8f654-pc6nz     1/1     Running   0           33s   10.0.186.121    ip-10-0-181-187.ec2.internal       <none>            <none>
~ $

```

Figura. Recuperación post-redeploy: pods reemplazados tras un nuevo despliegue (rollout exitoso).

```
kubectl get pods -n tienda -o wide # antes/después: pods nuevos = rollout OK
```

Nota sobre la persistencia de datos (limitación conocida). El Deployment de MySQL usa un volumen emptyDir, atado al ciclo de vida del pod, por lo que la base de datos es **efímera**: cualquier reprogramación, borrado o redeploy del pod tienda-db la reinicia desde cero, conservando solo los productos semilla de la imagen y perdiendo los cambios hechos por CRUD en tiempo de ejecución. Como el pipeline también aplica set image a la DB en cada push, **la demostración del CRUD debe realizarse después del último despliegue**. El detalle y la mejora futura propuesta se desarrollan en la sección «Problemas encontrados y limitaciones».

Mejoras aplicadas y mejora continua

Sobre el proyecto base se aplicaron mejoras de estructura, seguridad y automatización:

- **Estructura:** se unificó todo en un **monorepo** (los repos entregados eran idénticos), de modo que GitHub Actions detecta el workflow; el material del profesor se archivó en docs_profesor/.
- **Código:** se corrigió server.js (DB_HOST sin comillas → referencia inválida).
- **Seguridad (IE5):** se quitaron contraseñas horneadas en db/Dockerfile y el mysql-secret.yaml versionado; ahora el Secret se genera en el pipeline desde GitHub Secrets.
- **Red (IE1/IE2):** diseño de producción (nodos privados + NAT + ELB en públicas) y etiquetado de subredes para el balanceo; se reemplazaron las anotaciones del AWS LB Controller (no disponible en Learner Lab) por un Classic ELB.
- **CI/CD (IE4):** pipeline que usa el Environment *production*, genera el Secret de MySQL, despliega también la imagen de la DB y etiqueta imágenes con SHA + latest; el Metrics Server ya no se instala desde el pipeline (solo se verifica), eliminando el conflicto que dejaba el HPA en <unknown>.

Problemas encontrados y limitaciones

Problema	Solución
HPA en «» y kubectl top sin métricas.	Conflicto entre el metrics-server de upstream y el complemento del EKS; se quitó la instalación del pipeline y se dejó solo el complemento gestionado.
Credenciales del Learner Lab que rotan cada sesión.	Secrets en GitHub Environment + aws eks update-kubeconfig al inicio de cada sesión.
type: LoadBalancer requería AWS LB Controller (no disponible).	Se usó un Classic ELB (Service LoadBalancer sin anotaciones del controller).
Contraseñas versionadas en db/Dockerfile y mysql-secret.yaml.	Se eliminaron del repo; el Secret de MySQL se genera en el pipeline desde GitHub Secrets.

Limitación conocida: persistencia efímera de la base de datos

El Deployment de MySQL (k8s/mysql-deployment.yaml) monta un volumen emptyDir: {} en /var/lib/mysql. Un emptyDir está atado al **ciclo de vida del pod** —no del contenedor—, por lo que la base de datos es **efímera**: si el pod tienda-db se borra, se reprograma a otro nodo, o se realiza un redeploy/rollout (kubectl set image), el volumen se destruye y MySQL se reinicializa desde cero.

Tras un reinicio del pod solo se recuperan los productos «semilla» horneados en la imagen tienda-db (productos 1–5); se pierden los cambios realizados en tiempo de ejecución (altas, ediciones y bajas vía CRUD). La consecuencia operativa es relevante para la demo: dado que el pipeline CI/CD aplica `set image` también a la DB en cada push, **cada despliegue reinicia la base de datos**, por lo que el CRUD debe ejecutarse **después del último deploy**.

Esta decisión es **intencional** dado el contexto de laboratorio (AWS Academy Learner Lab), donde se priorizó la simplicidad y el ahorro de créditos por sobre la persistencia. La superación de esta limitación se detalla en la sección de Conclusiones y mejora continua.

5. Conclusión

La solución implementada cumple con los objetivos de negocio de Tienda Perritos y con los siete indicadores del encargo. La aplicación quedó orquestada en **AWS EKS** sobre una arquitectura de **producción** (2 AZ, nodos privados, ELB público) que aporta **alta disponibilidad** y **seguridad por defensa en capas**.

La **automatización** del ciclo de vida con GitHub Actions eliminó los despliegues manuales: un simple *commit* a **main** reconstruye, publica y despliega la aplicación en ~2 minutos, con **recuperación sin downtime** gracias a los *rollouts* de Kubernetes. La **escalabilidad** quedó demostrada con el HPA, que escaló el backend de 2 a 10 réplicas ante una carga real y volvió a su estado base al cesar la demanda. La **seguridad de credenciales** se resolvió sacando todo secreto del código y centralizándolo en un GitHub Environment, generando el Secret de Kubernetes en tiempo de despliegue.

En cuanto a **mejora continua**, la prioridad más relevante es **dotar de persistencia a la base de datos**, hoy efímera por el uso de `emptyDir`. La recomendación es reemplazar ese volumen por un **PersistentVolumeClaim (PVC)** respaldado por **Amazon EBS** —lo que requiere instalar el addon `aws-ebs-csi-driver`— y, idealmente, migrar el Deployment de MySQL a un **StatefulSet**, que aporta identidad de red y almacenamiento estables para cargas con estado. Como alternativa de producción, lo más robusto sería **desacoplar la base de datos del clúster** empleando un servicio gestionado como **Amazon RDS for MySQL**.

Otros pasos naturales son: incorporar un **Cluster Autoscaler** para escalar también los nodos EC2; añadir **HTTPS/Ingress** con un certificado ACM y un dominio propio en Route 53; sumar **pruebas automatizadas** al pipeline antes del deploy; y desplegar un **NAT Gateway por AZ** para eliminar ese punto único de fallo en un entorno realmente productivo. En conjunto, el proyecto evidencia competencias integrales de DevOps: automatizar, desplegar y optimizar software en un entorno de nube real aplicando buenas prácticas profesionales.